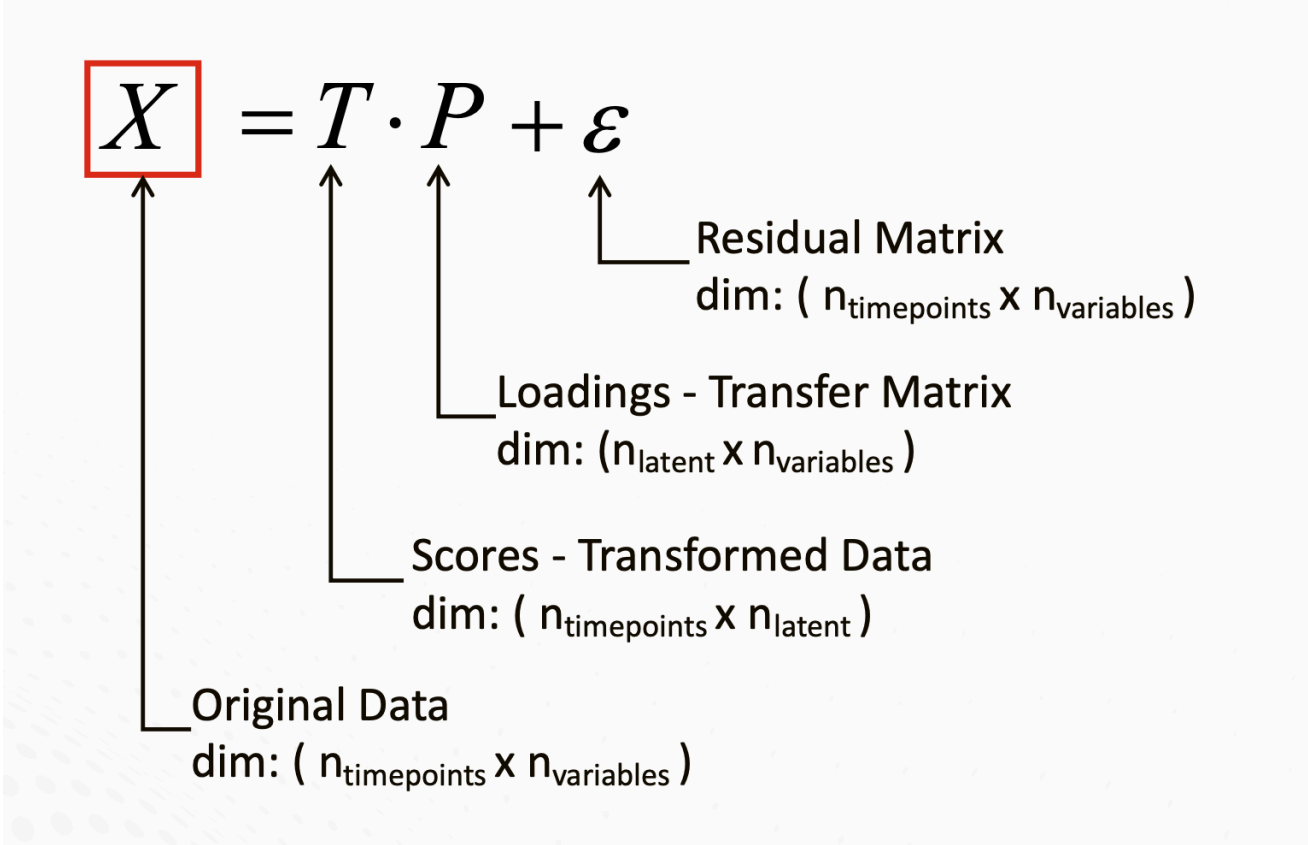
Notebook setup

Run the next cell first. In Google Colab it clones the course repository, then imports the libraries and the course modules. If you cloned the repository and run the notebook locally (see the README), it only does the imports.

Principal Component Analysis

Principal Component Analysis (PCA) finds the underlying dimension of a dataset that suffices to capture the information in the data, the so-called latent variables.

We want to compress the information into a defined number of latent variables, such that `n_latent` $<$ `n_variables`. Each principal component is a linear combination of the original variables (the **loadings**), and each observation gets a coordinate on every component (the **scores**).



Food consumption data

We use PCA to explore the differences between European countries in their food consumption habits. Each row is a country and each column the percentage of households consuming a food item.

Source: ProSensus course, Basel, 2014. Dataset available at http://openmv.net/info/food-consumption (a copy is stored in `datasets/food-consumption.csv`).

```
# Load the dataset (a copy of http://openmv.net/file/food-consumption.csv)
food_cons_na = pd.read_csv("datasets/food-consumption.csv", index_col=0)
food_cons_na
```

Country	Real coffee	Instant coffee	Tea	Sweetener	Biscuits	Powder soup	Tin soup	Potatoes	Frozen fish	Frozen veggies	Apples	Oranges	Tinned fruit	Jam	Garlic	Butter	Margarine	Olive oil	Yoghurt	Crisp bread
Germany	90	49	88	19.0	57.0	51	19	21	27	21	81	75	44	71	22	91	85	74	30.0	26
Italy	82	10	60	2.0	55.0	41	3	2	4	2	67	71	9	46	80	66	24	94	5.0	18
France	88	42	63	4.0	76.0	53	11	23	11	5	87	84	40	45	88	94	47	36	57.0	3
Holland	96	62	98	32.0	62.0	67	43	7	14	14	83	89	61	81	15	31	97	13	53.0	15
Belgium	94	38	48	11.0	74.0	37	23	9	13	12	76	76	42	57	29	84	80	83	20.0	5
Luxembourg	97	61	86	28.0	79.0	73	12	7	26	23	85	94	83	20	91	94	94	84	31.0	24
England	27	86	99	22.0	91.0	55	76	17	20	24	76	68	89	91	11	95	94	57	11.0	28
Portugal	72	26	77	2.0	22.0	34	1	5	20	3	22	51	8	16	89	65	78	92	6.0	9
Austria	55	31	61	15.0	29.0	33	1	5	15	11	49	42	14	41	51	51	72	28	13.0	11
Switzerland	73	72	85	25.0	31.0	69	10	17	19	15	79	70	46	61	64	82	48	61	48.0	30
Sweden	97	13	93	31.0	NaN	43	43	39	54	45	56	78	53	75	9	68	32	48	2.0	93
Denmark	96	17	92	35.0	66.0	32	17	11	51	42	81	72	50	64	11	92	91	30	11.0	34
Norway	92	17	83	13.0	62.0	51	4	17	30	15	61	72	34	51	11	63	94	28	2.0	62
Finland	98	12	84	20.0	64.0	27	10	8	18	12	50	57	22	37	15	96	94	17	NaN	64
Spain	70	40	40	NaN	62.0	43	2	14	23	7	59	77	30	38	86	44	51	91	16.0	13
Ireland	30	52	99	11.0	80.0	75	18	2	5	3	57	52	46	89	5	97	25	31	3.0	9

What are the options to deal with missing data?

Some entries of the table are missing (`NaN`). Common options are:

- remove the rows (countries) or columns (food items) with missing values,
- replace the missing values with a summary statistic such as the column mean or median,
- estimate the missing values with a model (for example an iterative PCA imputation).

Here we take the simplest option and replace missing values with the column mean.

```
food_cons = food_cons_na.fillna(food_cons_na.mean())
food_cons
```

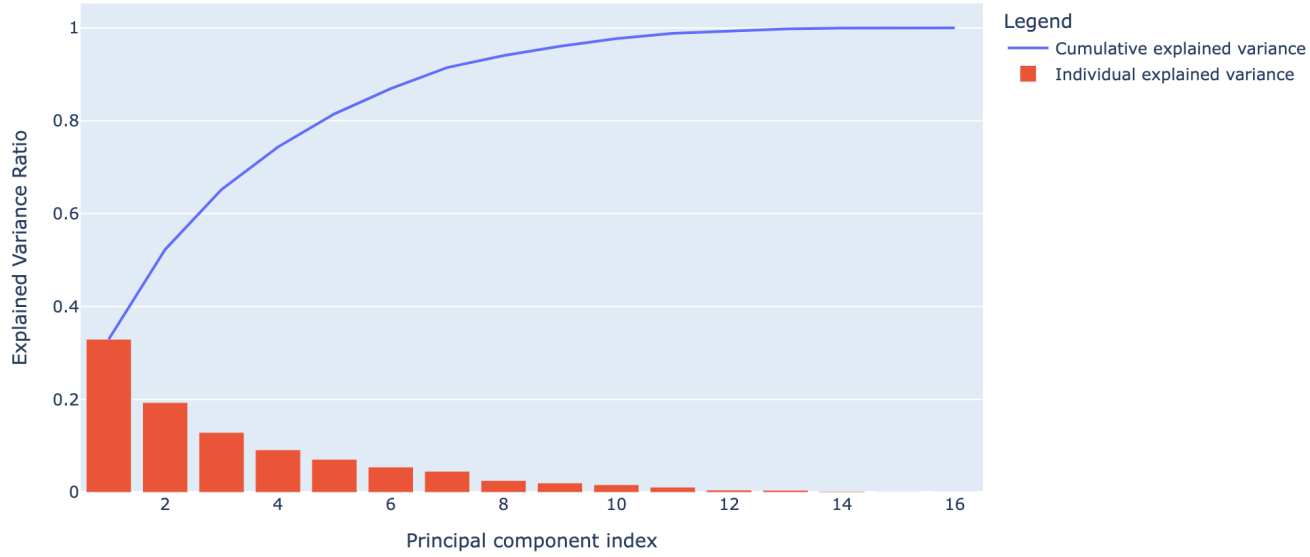
Country	Real coffee	Instant coffee	Tea	Sweetener	Biscuits	Powder soup	Tin soup	Potatoes	Frozen fish	Frozen veggies	Apples	Oranges	Tinned fruit	Jam	Garlic	Butter	Margarine	Olive oil	Yoghurt	Crisp bread
Germany	90	49	88	19.0	57.00	51	19	21	27	21	81	75	44	71	22	91	85	74	30.00	26
Italy	82	10	60	2.0	55.00	41	3	2	4	2	67	71	9	46	80	66	24	94	5.00	18
France	88	42	63	4.0	76.00	53	11	23	11	5	87	84	40	45	88	94	47	36	57.00	3
Holland	96	62	98	32.0	62.00	67	43	7	14	14	83	89	61	81	15	31	97	13	53.00	15
Belgium	94	38	48	11.0	74.00	37	23	9	13	12	76	76	42	57	29	84	80	83	20.00	5
Luxembourg	97	61	86	28.0	79.00	73	12	7	26	23	85	94	83	20	91	94	94	84	31.00	24
England	27	86	99	22.0	91.00	55	76	17	20	24	76	68	89	91	11	95	94	57	11.00	28
Portugal	72	26	77	2.0	22.00	34	1	5	20	3	22	51	8	16	89	65	78	92	6.00	9
Austria	55	31	61	15.0	29.00	33	1	5	15	11	49	42	14	41	51	51	72	28	13.00	11
Switzerland	73	72	85	25.0	31.00	69	10	17	19	15	79	70	46	61	64	82	48	61	48.00	30
Sweden	97	13	93	31.0	60.67	43	43	39	54	45	56	78	53	75	9	68	32	48	2.00	93
Denmark	96	17	92	35.0	66.00	32	17	11	51	42	81	72	50	64	11	92	91	30	11.00	34
Norway	92	17	83	13.0	62.00	51	4	17	30	15	61	72	34	51	11	63	94	28	2.00	62
Finland	98	12	84	20.0	64.00	27	10	8	18	12	50	57	22	37	15	96	94	17	20.53	64
Spain	70	40	40	18.0	62.00	43	2	14	23	7	59	77	30	38	86	44	51	91	16.00	13
Ireland	30	52	99	11.0	80.00	75	18	2	5	3	57	52	46	89	5	97	25	31	3.00	9

Score and loading plot of PCA

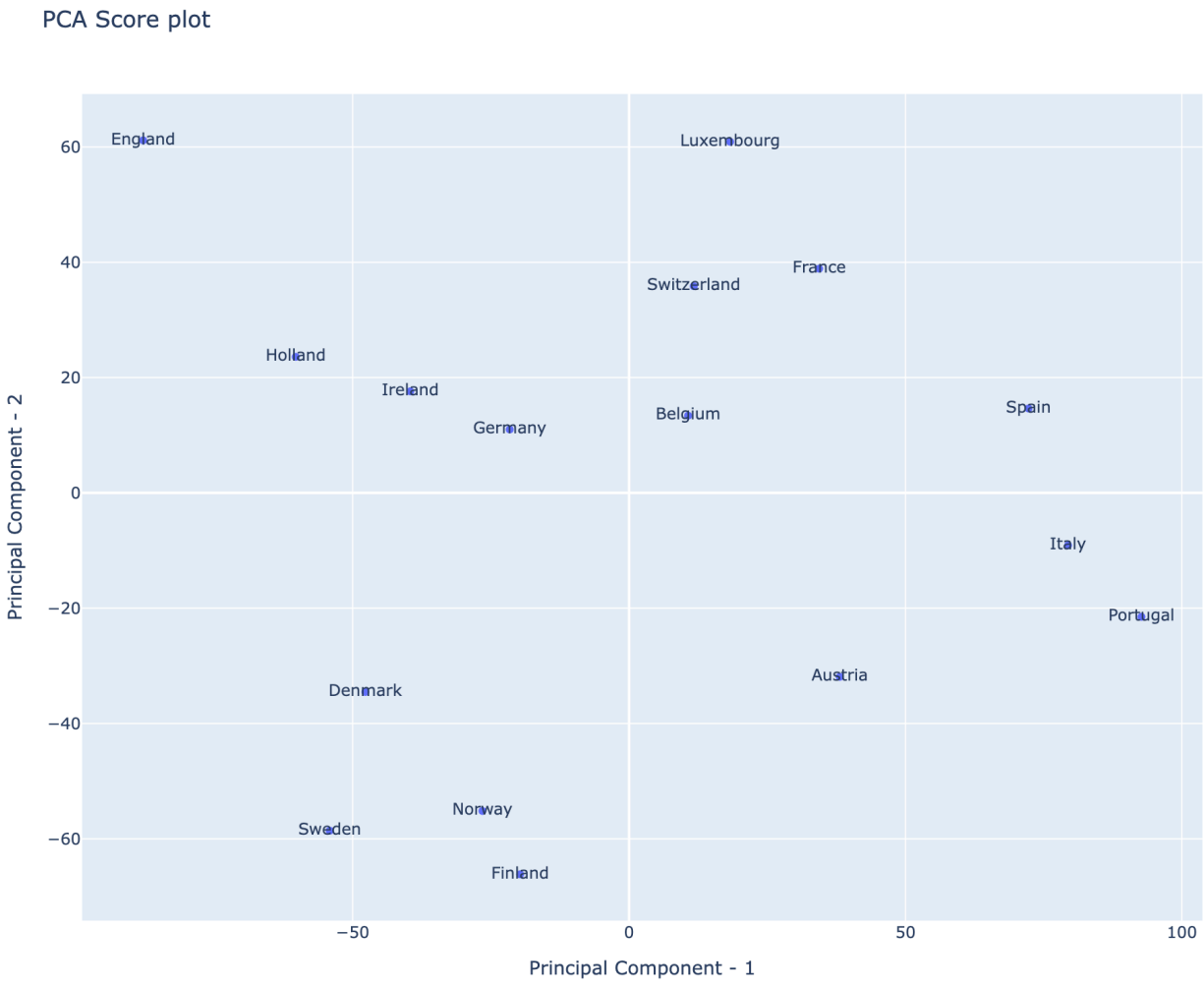
First we look at how much variance each principal component explains, then we plot the scores (countries) and the loadings (food items).

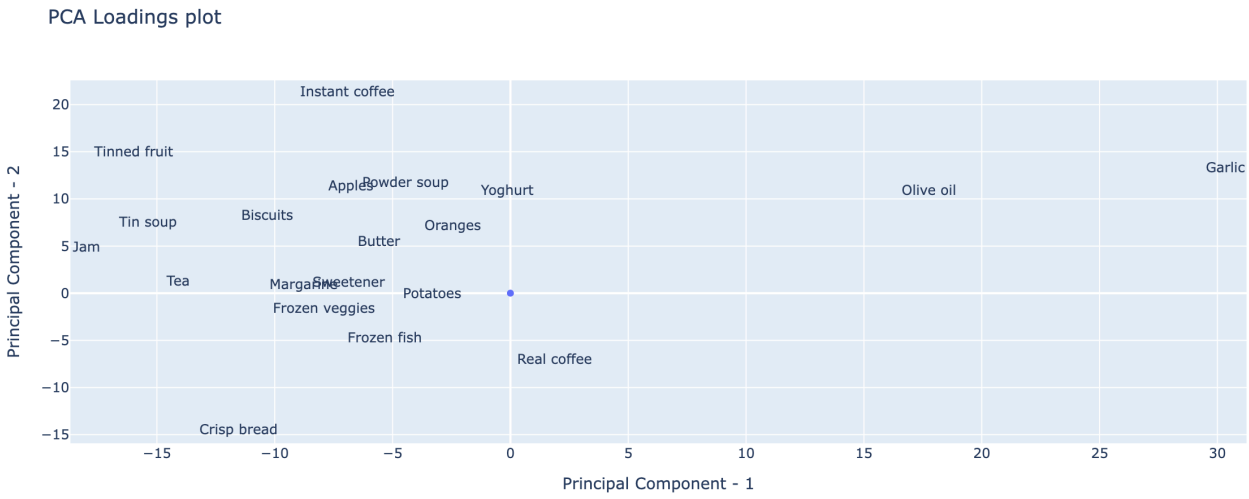
1. Why do we see clusters of countries in the score plot?
2. How can we understand the differences between these clusters?

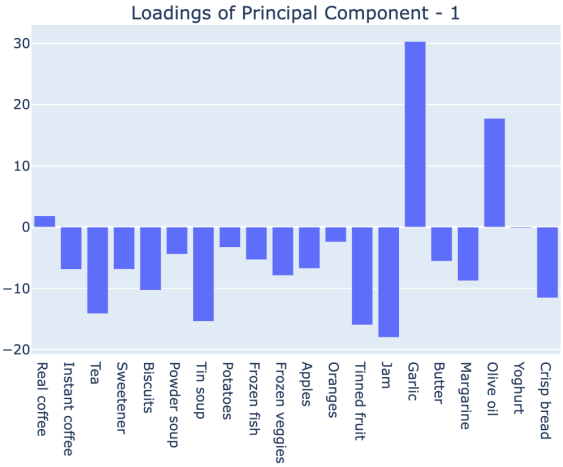
```
# Run PCA on the (mean centred) data
foodpca = PCA()
food_components = foodpca.fit_transform(food_cons)
foodpca_x = list(range(1, foodpca.n_components_ + 1))
```

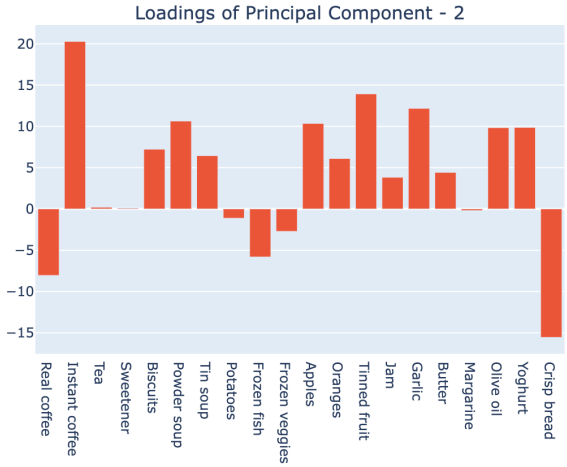



```
""" Principal component on x-axis and y-axis """
PC_X_AXIS = 1
PC_Y_AXIS = 2
```









Import or generate the in silico dataset

At the beginning of each notebook you decide whether to import the pre-generated experiments or to generate your own dataset with the in silico process model. For details on the process simulator, see the notebook `00_Process_Introduction.ipynb`.

- To import the dataset set `DATASET_SOURCE = "import"`.
- To generate a new dataset set `DATASET_SOURCE = "generate"`. The files are written to the folder `generated/`.

When running the notebook for the first time, we recommend the "import" option.

```
""" Select dataset source, one of (import, generate) """
DATASET_SOURCE = "import"
```

```
if DATASET_SOURCE == "import":
    # Define filename to import
    FILENAME = "owu.csv"
    FILEPATH = "datasets/"
    # Import OWU data
    owu = pd.read_csv(FILEPATH+FILENAME,index_col=None, usecols = ["X:VCD", "X:Glc",
"X:Lac", "X:Titer","W:Feed"])
    owu.index =
pd.MultiIndex.from_product([list(range(int(len(owu)/15))),list(range(15))], names=
["run","time"])
    # Import DOE data
    doe = pd.read_csv(FILEPATH+FILENAME.replace(".csv","_doe.csv"),index_col=None,
usecols =["feed_start","feed_end","Glc_feed_rate","Glc_0","VCD_0"])
```

```
if DATASET_SOURCE == "generate":
    # DOE Dataset definition (variable = [lower bound, upper bound])
    # Model parameters: Dictate the behaviour of the cell process
    MU_G_MAX = 0.05;
    MU_D_MAX = 0.025;
    K_G_GLC  = 1;
    K_I_LAC  = 30;
    K_D_LAC  = 50;
    K_GLC    = 0.04;
    K_LAC    = 0.06;
    K_PROD   = 1;
    # Process parameters: Conditions at which process is run
    FEED_START = [1, 4]
    FEED_END = [8, 12]
    GLC_FEED_RATE = [5, 20]
    GLC_0 = [10, 80.0]
    VCD_0 = [0.1, 1.0]
    # Number of experiments to generate
    NUM_RUNS = 40
    # Filename and filepath for the dataset
    FILENAME = "owu.csv"
    FILEPATH = "generated/"
    # Collect parameters to dictionary
    VAR_LIMS = {"mu_g_max":MU_G_MAX, "mu_d_max": MU_D_MAX, "K_g_Glc" : K_G_GLC,
"K_I_Lac" : K_I_LAC, "K_d_Lac" : K_D_LAC, "k_Glc" : K_GLC, "k_Lac" : K_LAC, "k_Prod"
: K_PROD,
        "feed_start" : FEED_START,"feed_end" : FEED_END, "Glc_feed_rate" :
GLC_FEED_RATE, "Glc_0" : GLC_0, "VCD_0" : VCD_0}
    # Generate Dataset
    data = simulator.generate_data(VAR_LIMS, NUM_RUNS, FILEPATH + FILENAME)
    # Import DOE
    doe = pd.read_csv(FILEPATH+FILENAME.replace(".csv","_doe.csv"),index_col=None,
usecols =["feed_start","feed_end","Glc_feed_rate","Glc_0","VCD_0"])
    # Import OWU
    owu = pd.read_csv(FILEPATH+FILENAME,index_col=None, usecols = ["X:VCD", "X:Glc",
"X:Lac", "X:Titer","W:Feed"])
    owu.index = pd.MultiIndex.from_product([list(range(NUM_RUNS)),list(range(15))],
names=["run","time"])
```

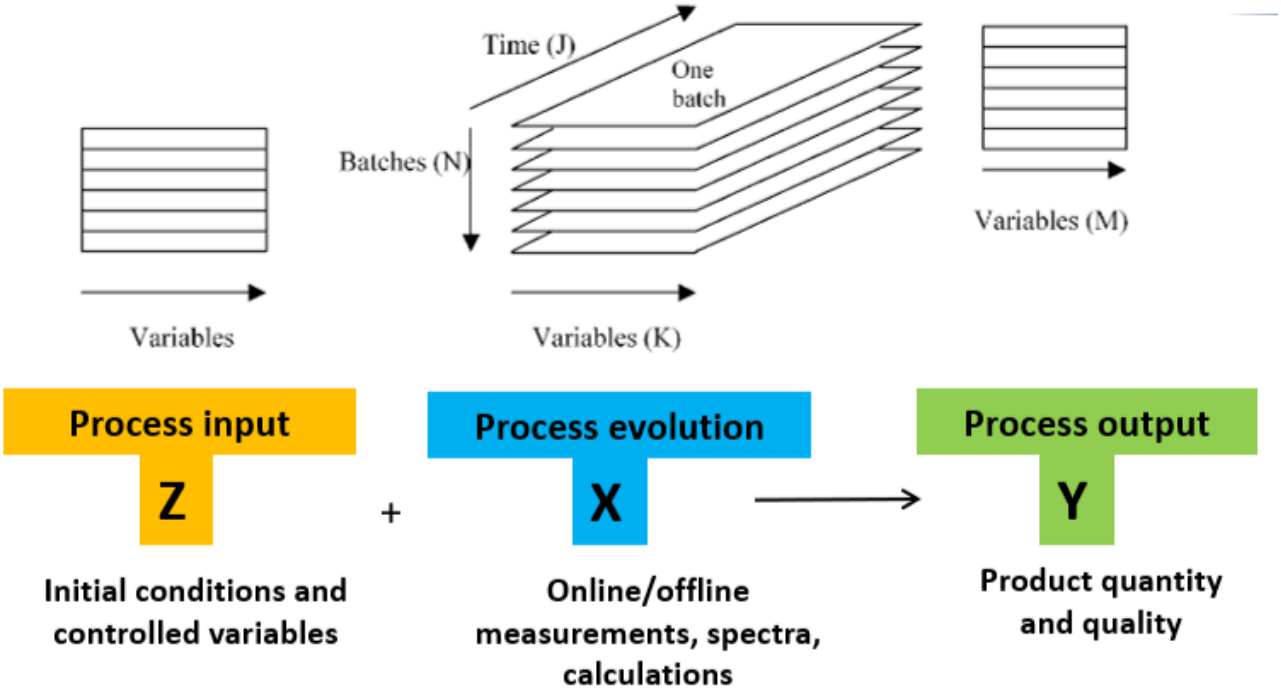
Exploratory analysis via PCA

- Why is it important to visualize and understand the data not only in a univariate but also in a multivariate setting?
- What insights can multivariate analysis provide?
- What conclusions can be made from PCA on the OWU matrix?
- What conclusions can be made from PCA on the BWU matrix?

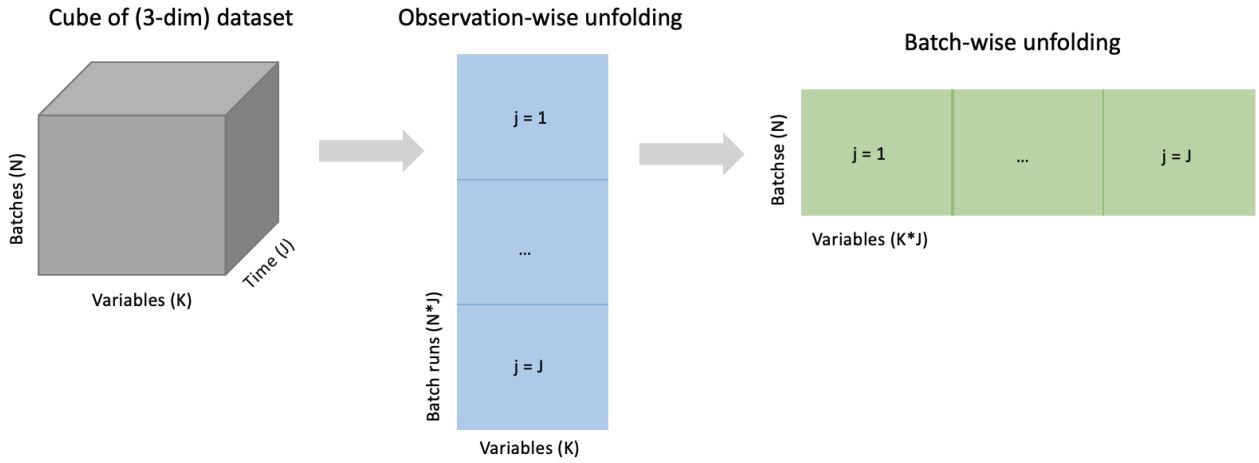
OWU and BWU Unfolding

First we need to distinguish between different types of variables and different types of unfolding.

Variable Types:



Unfolding types:



PCA of the OWU matrix

In this section we use the experiments generated with a Latin hypercube design (LHD) in the previous notebook. They are arranged in an observation-wise unfolded (OWU) matrix, where every row is one measurement time point of one run, and analysed with PCA.

Visualize the OWU matrix

In the OWU matrix, the 1st column corresponds to VCD, the 2nd to glucose, the 3rd to lactate, the 4th to titer, and the 5th to the feed rate.

```
owu
```

run	time	X:VCD	X:Glc	X:Lac	X:Titer	W:Feed
0	0	0.55	45.00	0.00	0.00	0.0
	1	1.73	44.01	1.49	0.05	0.0
	2	4.78	41.09	5.87	1.31	12.5
	3	10.28	40.27	16.49	15.87	12.5
	4	15.89	40.08	35.54	85.82	12.5
...	...	...	...	...	...	...
49	10	6.86	66.96	146.18	1078.53	0.0
	11	4.43	73.79	154.25	1171.21	0.0
	12	2.80	70.39	159.35	1230.61	0.0
	13	1.81	68.22	162.62	1268.88	0.0
	14	1.18	66.80	164.74	1293.89	0.0

750 rows × 5 columns

In the DOE matrix, the 1st column corresponds to feed_start, the 2nd to feed_end, the 3rd to Glc_feed_rate, the 4th to initial Glc, and the 5th to the initial VCD.

```
doe
```

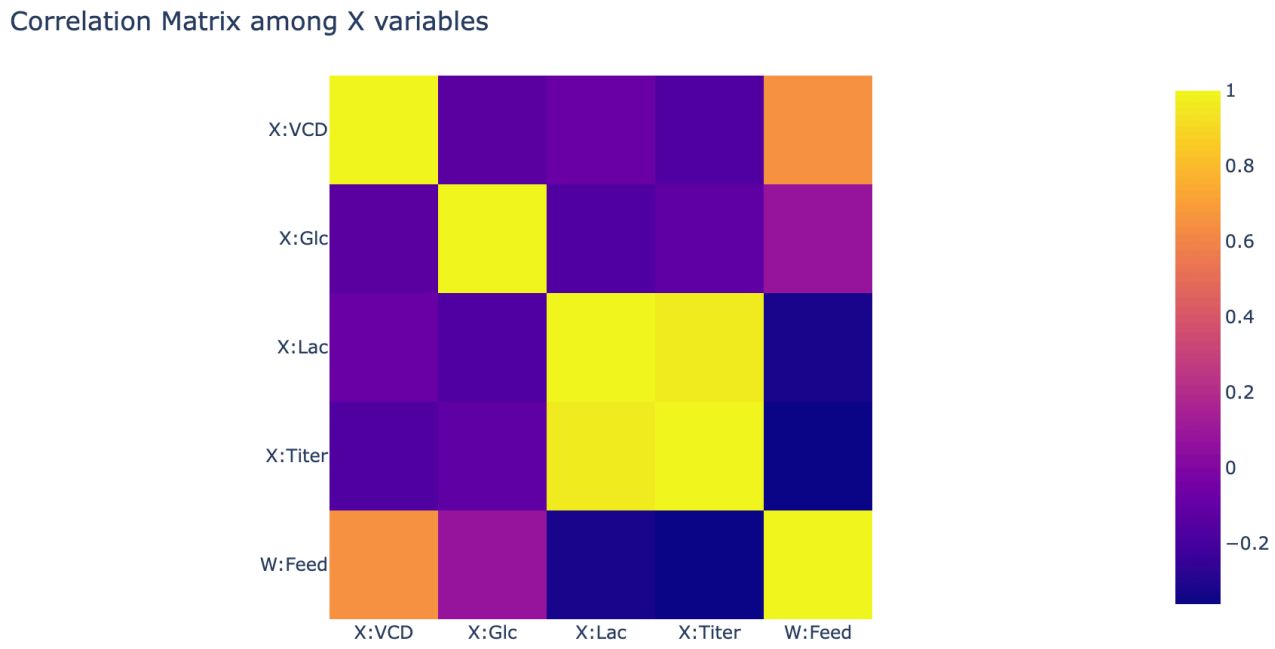
	feed_start	feed_end	Glc_feed_rate	Glc_0	VCD_0
0	2.50	10.00	12.50	45.00	0.55
1	1.95	10.24	18.62	40.71	0.27
2	3.30	9.35	14.64	17.86	0.13
3	3.48	9.02	15.56	47.86	0.84
4	1.46	9.59	7.30	33.57	0.42
...	...	...	...	...	...
45	2.38	8.61	6.99	59.29	0.51
46	2.68	8.78	10.36	57.86	0.20
47	3.17	10.00	8.52	63.57	0.70
48	2.56	10.08	6.07	39.29	0.72
49	1.83	10.82	14.95	42.14	0.35

50 rows × 5 columns

Plot correlation matrix

The OWU matrix is used to plot the degree of correlation between the different variables.

```
fig = px.imshow(owu.corr())
fig.update_layout(title='Correlation Matrix among X variables')
fig.show()
```

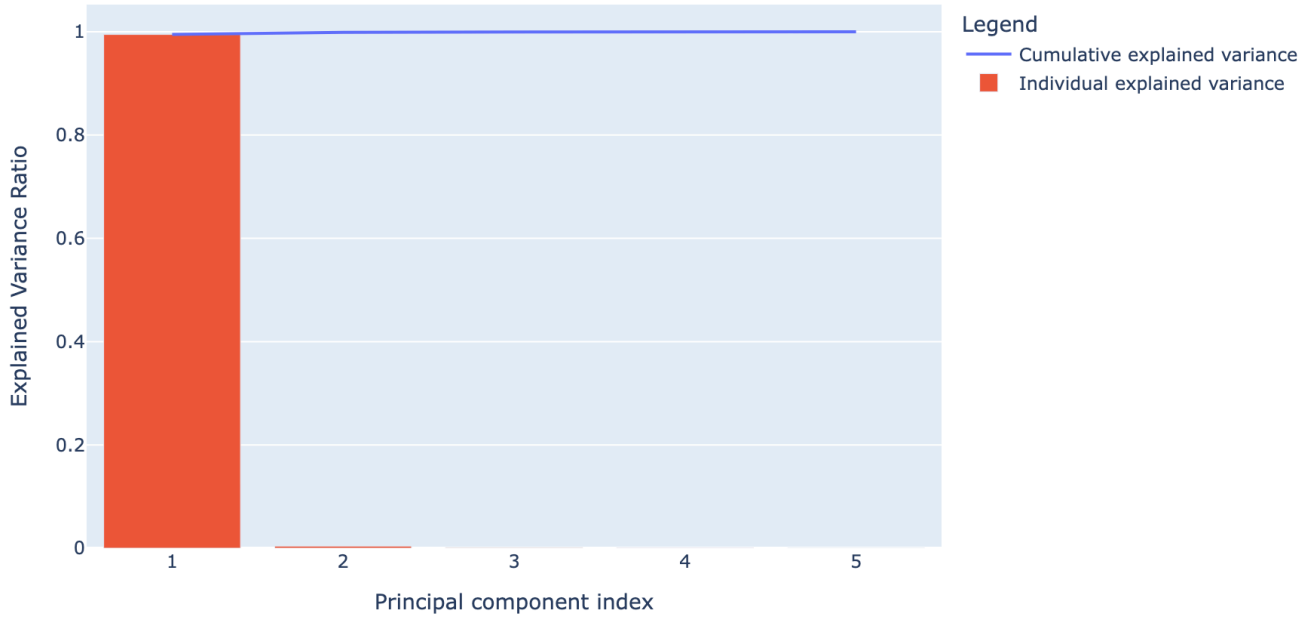


Unnormalized PCA

PCA is run on the OWU matrix without any variable normalization.

```
""" Number of components for PCA"""
PCA_N_COMPONENTS = 5
```

```
# Run PCA
owupca = PCA(n_components = PCA_N_COMPONENTS)
owupca.fit(owu)
owupca_x = list(range(1,owupca.n_components_+1))
```

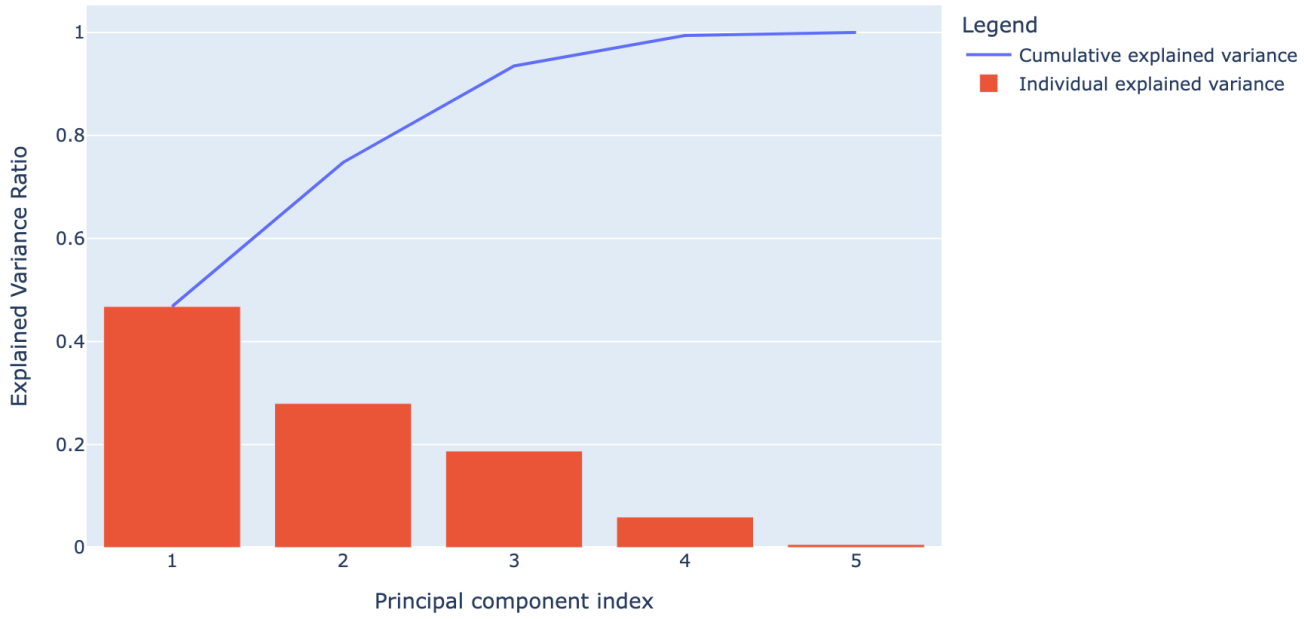


We can explain almost all the variance with the first principal component. Is that correct? Is something else incorrect about this analysis? (Hint: look at the magnitude of the variables in the OWU matrix.)

Normalized PCA

PCA is run on the OWU matrix, but this time the variables are first normalized with respect to their mean and standard deviation. This ensures that each variable contributes equally to the variance explained by the principal components.

```
# Scale data by mean and standard deviation
scaler = StandardScaler()
owu_normalized = scaler.fit_transform(owu)
# Run PCA on scaled data
owu_components = owupca.fit_transform(owu_normalized)
owupca_x = list(range(1, owupca.n_components_ + 1))
```



Plot scores and loadings

In the following plots, the PCA scores and the loadings are shown.

You can choose which principal components are displayed on which axis by changing `PC_X_AXIS` and `PC_Y_AXIS`.

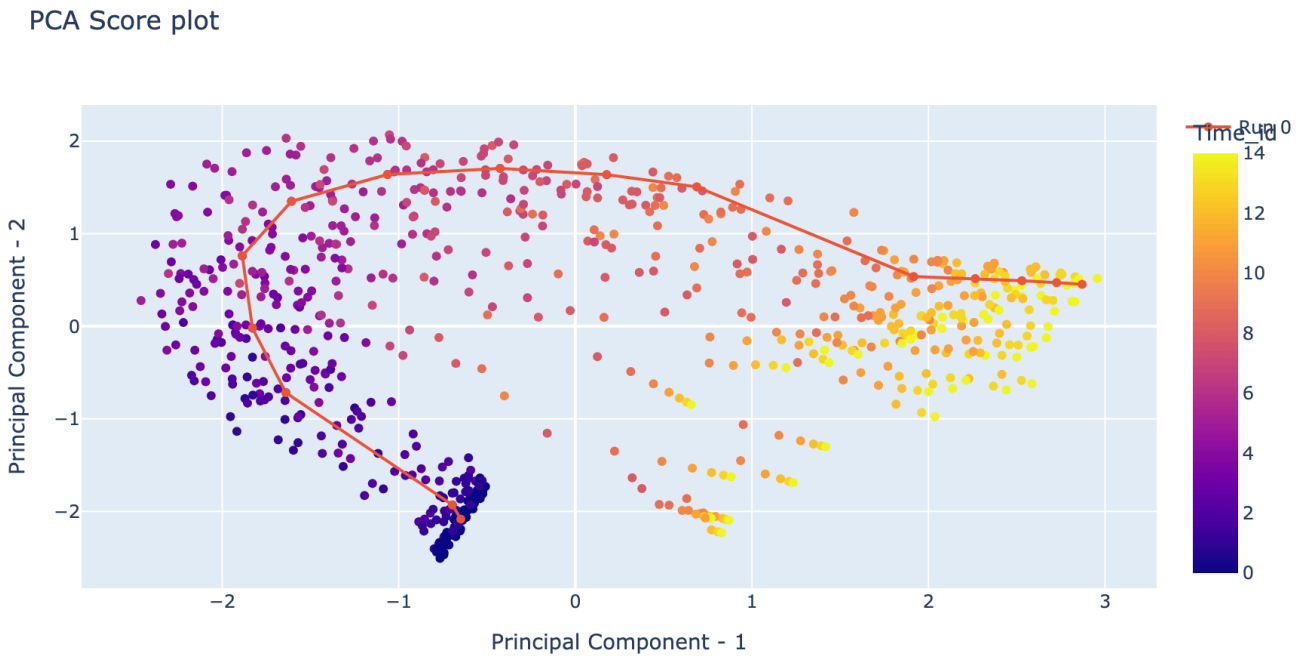
The points of the score plot can be coloured like in the previous notebook. The options for `PLOT_COLOR` are:

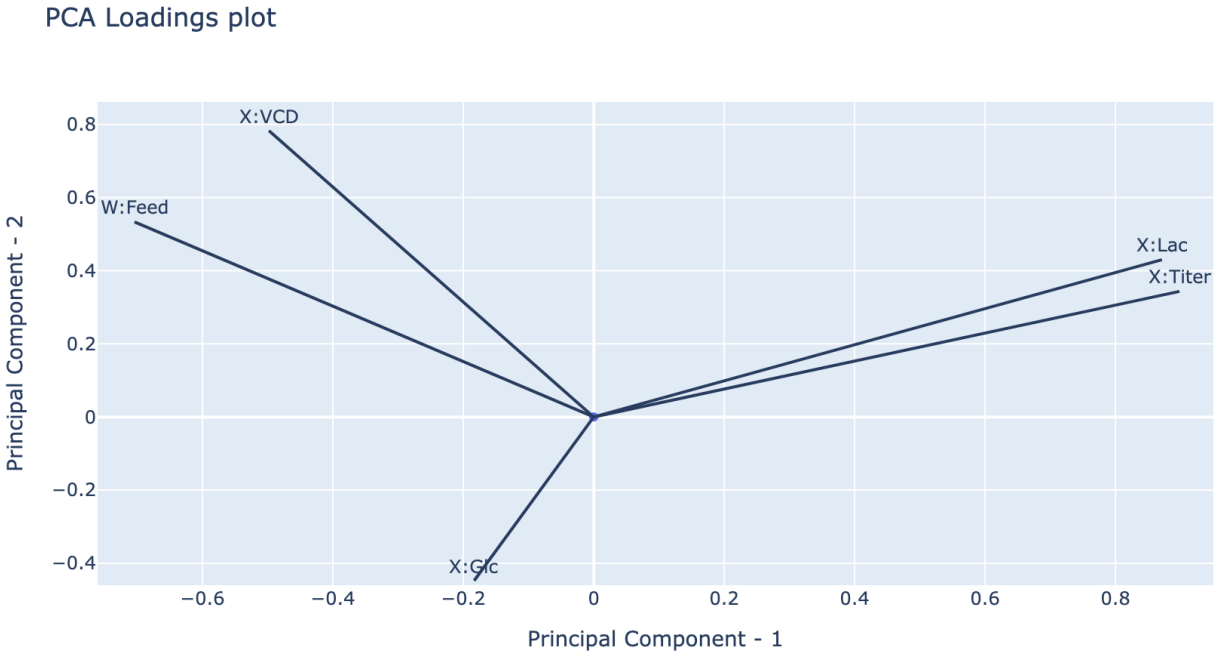
- `Run_id` points are coloured by the run they belong to.
- `Time_id` points are coloured by the day of the cell culture process.
- `Titer_14` points are coloured by the titer at day 14 of their run.

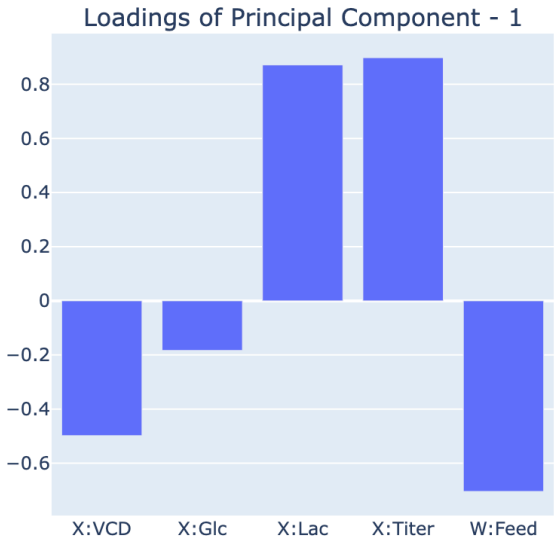
- `Glc_0` points are coloured by the designed initial glucose level.
- `VCD_0` points are coloured by the designed initial VCD level.
- `feed_start` points are coloured by the designed feed start day.
- `feed_end` points are coloured by the designed feed end day.
- `Glc_feed_rate` points are coloured by the designed glucose feed rate.

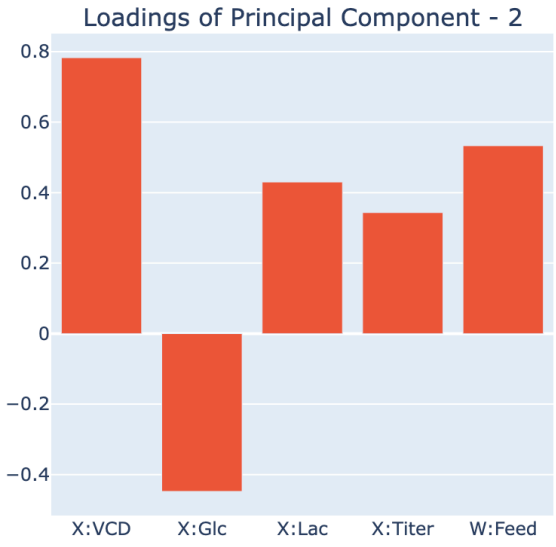
With `PLOT_RUN_ID` the trajectory of one run is highlighted in the score plot.

```
""" Principal component on x-axis and y-axis """
PC_X_AXIS = 1
PC_Y_AXIS = 2
""" Color points in plot by variable, one of (Titer_14,Time_id, Run_id, feed_start,
feed_end, Glc_feed_rate, Glc_0, VCD_0)"""
PLOT_COLOR = "Time_id"
""" Highlight specific run in plot """
PLOT_RUN_ID = 0
```

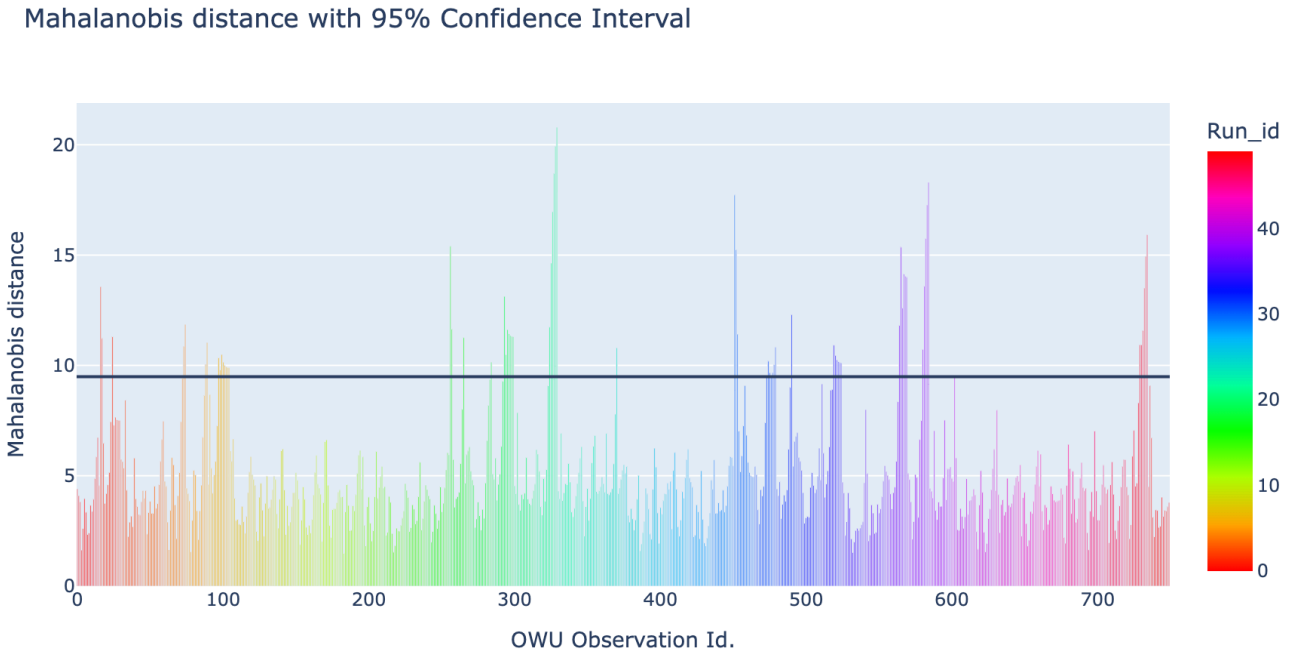









```
# Mahalanobis distance of the observations in the PCA score space, with the 95% chi-squared threshold (Hotelling's T2)
owu_mahalanobis =
EmpiricalCovariance().fit(owu_components).mahalanobis(owu_components)
T2threshold = chi2.ppf(0.95, owu_components.shape[1]-1)
fig = px.bar(x=list(range(len(owu_mahalanobis))),y=owu_mahalanobis,
labels=dict(x="OWU Observation Id.", y="Mahalanobis distance", color="Run_id"),
title="Mahalanobis distance with 95% Confidence Interval",
color=owu.index.get_level_values(0), color_continuous_scale=px.colors.cyclical.HSV)
fig.add_hline(y=T2threshold)
fig.show()
```



PCA of the BWU matrix

In the batch-wise unfolded (BWU) matrix, every row is one experiment run and each variable like `X:VCD` is unfolded into one column per time step: `X:VCD:0`, `X:VCD:1`, ..., `X:VCD:14`.

Visualize the BWU matrix

In the BWU matrix, the 1st column corresponds to VCD at day 0, the 2nd to glucose at day 0, the 3rd to lactate at day 0, the 4th to titer at day 0, and the 5th to the feed rate at day 0. This is repeated for day 1 and the following days. So, column 6 corresponds to VCD at day1, column 7 corresponds to Glc at day1, ..., column 11 corresponds to VCD at day2, etc.

```
# Transform OWU to BWU
bwu = simulator.generate_bwu(owu)
target = simulator.generate_y(bwu)
```

```
# Visualize BWU matrix
bwu
```

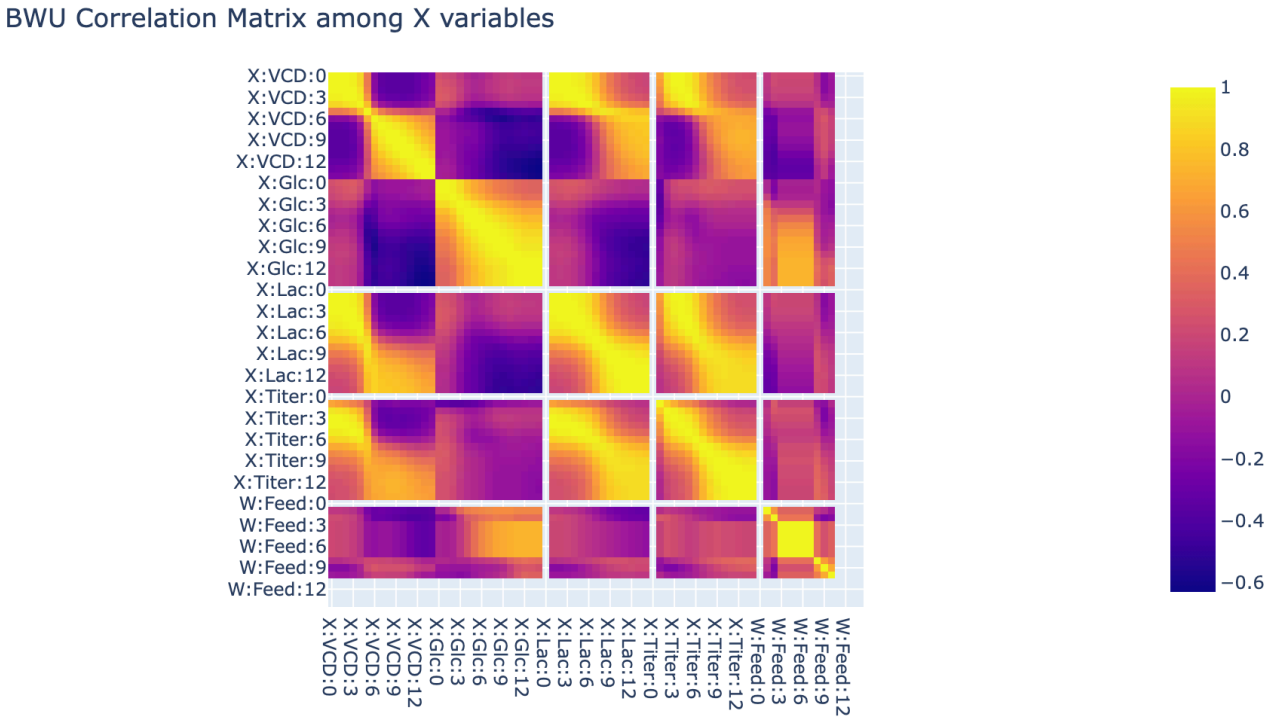
	X:VCD:0	X:VCD:1	X:VCD:2	X:VCD:3	X:VCD:4	X:VCD:5	X:VCD:6	X:VCD:7	X:VCD:8	X:VCD:9	...	W:Feed:5	W:Feed:6	W:Feed:7	W:Feed:8	W:Feed:9	W:Feed:10	W:Feed:11	W:Feed:
0	0.55	1.73	4.78	10.28	15.89	18.57	18.10	15.81	12.92	10.12	...	12.50	12.50	12.50	12.50	12.50	0.0	0.0	
1	0.27	0.87	2.59	6.53	12.16	15.86	15.66	12.76	8.91	5.46	...	18.62	18.62	18.62	18.62	18.62	0.0	0.0	
2	0.13	0.39	1.18	3.25	7.42	12.96	16.94	17.86	16.42	13.88	...	14.64	14.64	14.64	14.64	0.00	0.0	0.0	
3	0.84	2.61	6.77	12.92	17.67	18.96	17.51	14.78	11.82	9.13	...	15.56	15.56	15.56	15.56	0.00	0.0	0.0	
4	0.42	1.32	3.75	8.64	14.49	18.04	18.23	16.02	11.66	7.80	...	7.30	7.30	7.30	7.30	0.00	0.0	0.0	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
45	0.51	1.62	4.54	9.90	15.49	18.31	18.04	15.86	12.97	10.08	...	6.99	6.99	6.99	0.00	0.00	0.0	0.0	
46	0.20	0.65	1.97	5.25	10.67	15.56	17.52	16.82	14.66	12.02	...	10.36	10.36	10.36	0.00	0.00	0.0	0.0	
47	0.70	2.18	5.84	11.70	16.83	18.74	17.74	15.21	12.25	9.48	...	8.52	8.52	8.52	8.52	8.52	0.0	0.0	
48	0.72	2.22	5.89	11.78	16.89	18.59	16.02	10.83	7.29	5.10	...	6.07	6.07	6.07	6.07	6.07	0.0	0.0	
49	0.35	1.10	3.21	7.71	13.49	17.10	17.36	15.43	12.61	9.64	...	14.95	14.95	14.95	14.95	14.95	0.0	0.0	

50 rows × 75 columns

Plot correlation matrix

The BWU matrix is used to plot the degree of correlation between the different variables.

```
# Plot correlation of BWU matrix
fig = px.imshow(bwu.corr())
fig.update_layout(title='BWU Correlation Matrix among X variables')
fig.show()
```

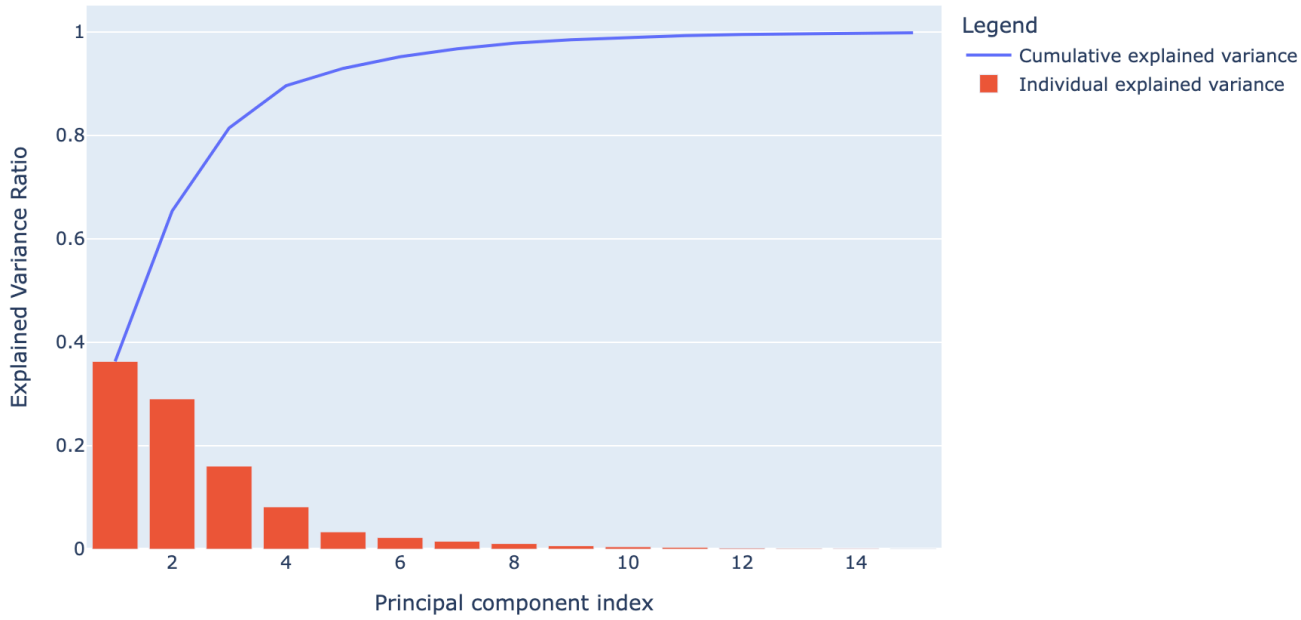


Normalized PCA

PCA is run on the BWU matrix. The variables are first normalized with respect to their mean and standard deviation. Note that the number of components is limited by the number of runs.

```
""" Number of components for PCA"""
PCA_N_COMPONENTS = 15
```

```
# Scale data by mean and standard deviation
scaler = StandardScaler()
bwu_normalized = scaler.fit_transform(bwu)
# Run PCA on scaled data
bwupca = PCA(n_components=min(min(bwu.shape),PCA_N_COMPONENTS))
bwupca.fit(bwu_normalized)
bwupca_x = list(range(1,bwupca.n_components_+1))
bwu_components = bwupca.fit_transform(bwu_normalized)
```

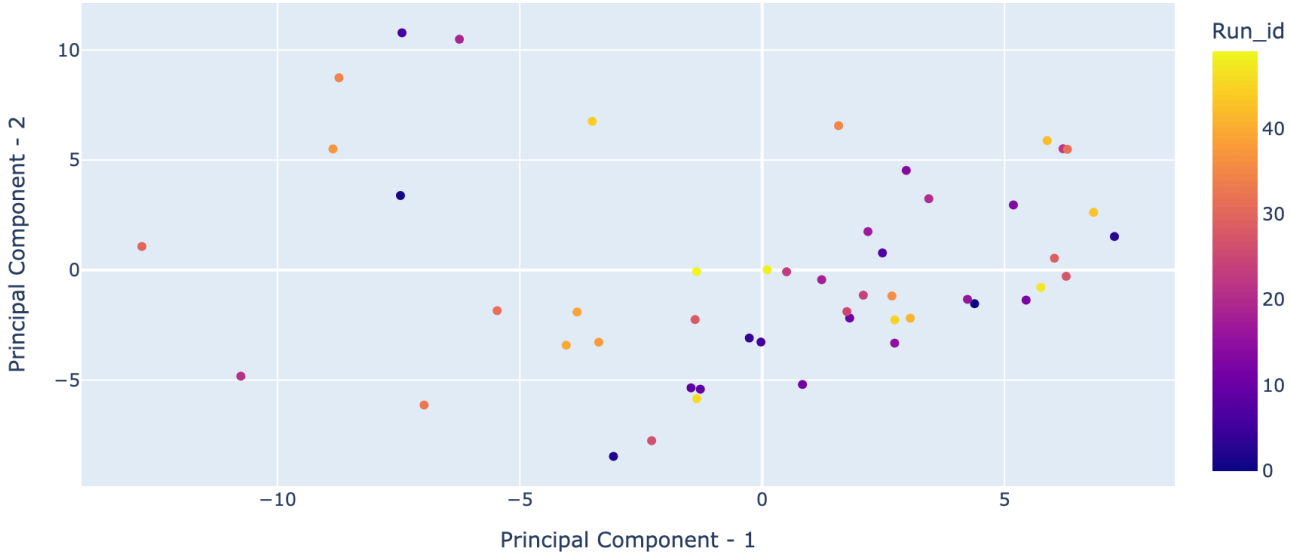


Plot scores and loadings

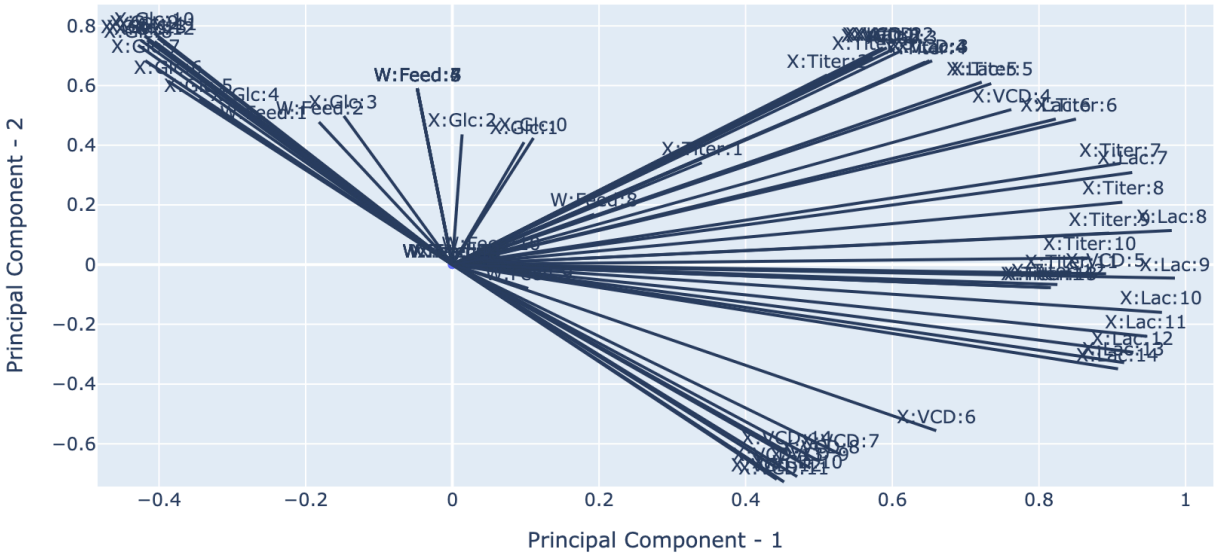
The scores (one point per run) are plotted first, followed by the loadings (one entry per variable and day). Compare the loadings of the same variable at different days.

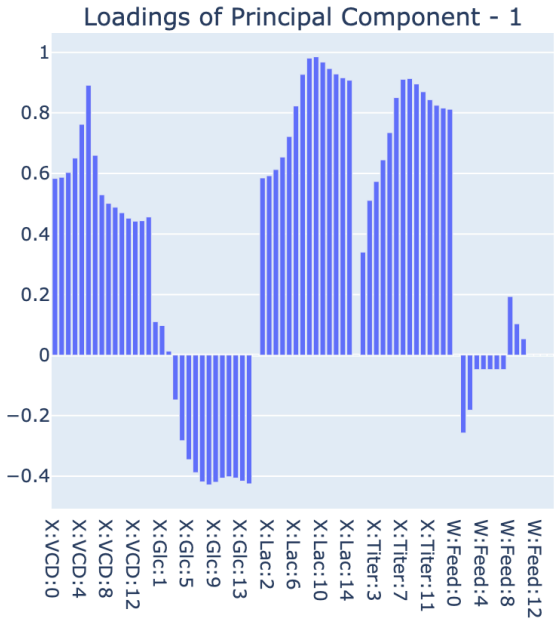
```
""" Principal component on x-axis and y-axis """
PC_X_AXIS = 1
PC_Y_AXIS = 2
""" Color points in plot by variable, one of (Titer_14, Run_id, feed_start, feed_end,
Glc_feed_rate, Glc_0, VCD_0)"""
PLOT_COLOR = "Run_id"
```

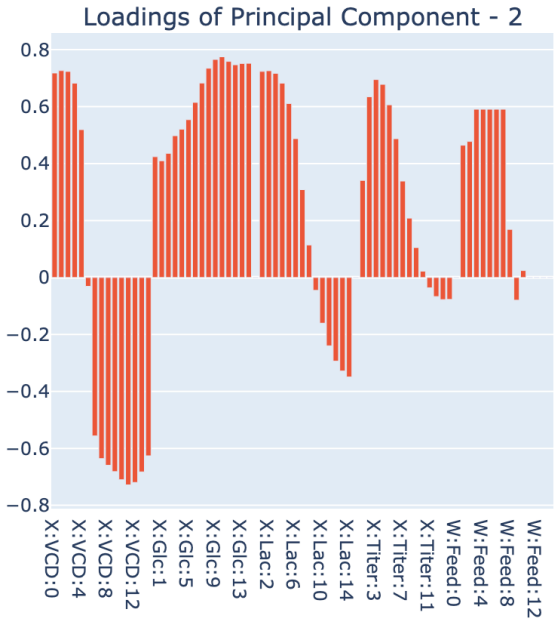
PCA BWU Score plot



PCA Loadings plot







Tasks

1. Compute the Mahalanobis distances of the runs in the BWU score space. Are there outlying runs?
2. How would you use the PCA analysis to find high titer experiments?
3. Normalize the data by the overall median VCD, the run-wise median VCD or the time-wise median VCD and run PCA on the OWU matrix (pick one). How do the scores change?

```
# Your code here
```